

APPLIED DATA STRUCTURES & ALGORITHMS LAB (BCS-553)

B.Tech CSE 5th SEMESTER

SUBMMITED TO:

MR. RASHAN RAJ

SUBMMITED BY:

MANKIRAT SINGH

ID: BCS2024542

ROLL NO: 2410303257

Lab Assignment 12: Recursive Factorial, Sum and Power

Experiment No. 12.1 - Recursive factorial

Experiment Title:

Recursive factorial

Aim:

To compute the factorial of a number using recursion.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Recursion is a technique in which a method calls itself on a smaller input. Every recursive method needs a base case, which stops the calls, and a recursive case, which reduces the problem. Each call is stored on the call stack, so the space used grows with the depth of the recursion. Without a base case the program ends with StackOverflowError.

Algorithm / Procedure:

1. Base case: if n == 0 return 1.
2. Recursive case: return n * factorial(n - 1).
3. Call factorial(5) from main and print the result.

Program / Queries / Commands:

```
// Name: Mankirat Singh
// ID: BCS2024542
// Roll No: 24103032571
// Assignment 12, Task 1
// Question: Write the method int factorial(int n) using recursion.
// The factorial of 0 is 1, and n times factorial(n - 1) is the other
// case. Print factorial(5).

public class Main {
    static int factorial(int n) {
        if (n == 0) {
            return 1;
        }
        return n * factorial(n - 1);
    }

    public static void main(String[] args) {
        System.out.println(factorial(5));
    }
}
```

Input:

n = 5.

Output:

```
120
```

Time / Space Complexity: Time O(n); Space O(n) for the call stack.

Result:

The program was compiled and executed successfully. factorial(5) was computed recursively and equals 120.

Viva-Voce Questions:

- What is recursion?
- What is a base case?
- What is the factorial of 0?

Experiment No. 12.2 - Recursive sum of 1 to n

Experiment Title:

Recursive sum of 1 to n

Aim:

To add the numbers from 1 to n using recursion.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Recursion is a technique in which a method calls itself on a smaller input. Every recursive method needs a base case, which stops the calls, and a recursive case, which reduces the problem. Each call is stored on the call stack, so the space used grows with the depth of the recursion. Without a base case the program ends with StackOverflowError.

Algorithm / Procedure:

1. Base case: if n == 0 return 0.
2. Recursive case: return n + sum(n - 1).
3. Call sum(10) from main and print the result.

Program / Queries / Commands:

```
// Name: Mankirat Singh
// ID: BCS2024542
// Roll No: 24103032571
// Assignment 12, Task 2
// Question: Write the method int sum(int n) using recursion that
// adds the numbers from 1 to n. Print sum(10).

public class Main {
    static int sum(int n) {
        if (n == 0) {
            return 0;
        }
        return n + sum(n - 1);
    }

    public static void main(String[] args) {
        System.out.println(sum(10));
    }
}
```

Input:

n = 10.

Output:

```
55
```

Time / Space Complexity: Time O(n); Space O(n).

Result:

The program was compiled and executed successfully. sum(10) was computed recursively and equals 55.

Viva-Voce Questions:

- Why is the base case 0 here?
- What happens if the base case is missing?
- How can sum(n) be written without recursion?

Experiment No. 12.3 - Recursive power

Experiment Title:

Recursive power

Aim:

To compute base raised to exp using recursion.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Recursion is a technique in which a method calls itself on a smaller input. Every recursive method needs a base case, which stops the calls, and a recursive case, which reduces the problem. Each call is stored on the call stack, so the space used grows with the depth of the recursion. Without a base case the program ends with StackOverflowError.

Algorithm / Procedure:

1. Base case: if exp == 0 return 1.
2. Recursive case: return base * power(base, exp - 1).
3. Call power(2, 5) from main and print the result.

Program / Queries / Commands:

```
// Name: Mankirat Singh
// ID: BCS2024542
// Roll No: 24103032571
// Assignment 12, Task 3
// Question: Write the method int power(int base, int exp) using
// recursion. When exp is 0 the answer is 1; otherwise the answer is
// base times power(base, exp - 1). Print power(2, 5).

public class Main {
    static int power(int base, int exp) {
        if (exp == 0) {
            return 1;
        }
        return base * power(base, exp - 1);
    }

    public static void main(String[] args) {
        System.out.println(power(2, 5));
    }
}
```

Input:

base = 2, exp = 5.

Output:

```
32
```

Time / Space Complexity: Time O(exp); Space O(exp).

Result:

The program was compiled and executed successfully. power(2, 5) was computed recursively and equals 32.

Viva-Voce Questions:

- What is the base case of power()?
- How many recursive calls are made for exp = 5?
- What is the call stack?